



ULAB

UNIVERSITY OF LIBERAL ARTS
BANGLADESH

Submitted to:

Name: Mohammad Sultan Mahmud, PhD

Submitted by:

Name: Hridoy Hossain

ID:223014107

Department: cse

Hospital Management System Database Design

Problem Statement

This open-ended experiment's goal is to create a database for a hospital administration system. An Entity-Relationship (ER) model will be created, mapped to relational schemas, and the resultant tables will be normalised as part of the design process. Determining the limits of the system and making the required assumptions are part of the project's scope. A thorough explanation of the relational schemas, the ER model, and a step-by-step normalisation procedure from 1NF to BCNF will all be included in the final report.

Scope and Assumptions

Scope

The database will be the main place to keep track of important information about patients, doctors, nurses, receptionists, admissions, appointments, treatments, rooms, bills, and hospital departments. It will make important tasks easier, like keeping track of patient admissions and discharges, making appointments, and sending out bills for medical services. The ER model will include at least 10 different types of entities to meet the needs of the experiment. This will make sure that the hospital's main activities are fully represented.

Presumptions

In order to maintain the design's practicality and manageability, we have assumed the following:

Every patient will have a unique PatientID to distinguish them in the system.

Each doctor, nurse, and receptionist will be assigned a unique ID for easy identification.

An appointment will be booked for one patient with a single doctor at a time.

A treatment will be administered to one patient by a single doctor per record.

An admission will be treated as a single event for each patient's hospital stay.

Each room will be overseen by just one department to avoid confusion in management.

A receptionist will be assigned to work within a single department.

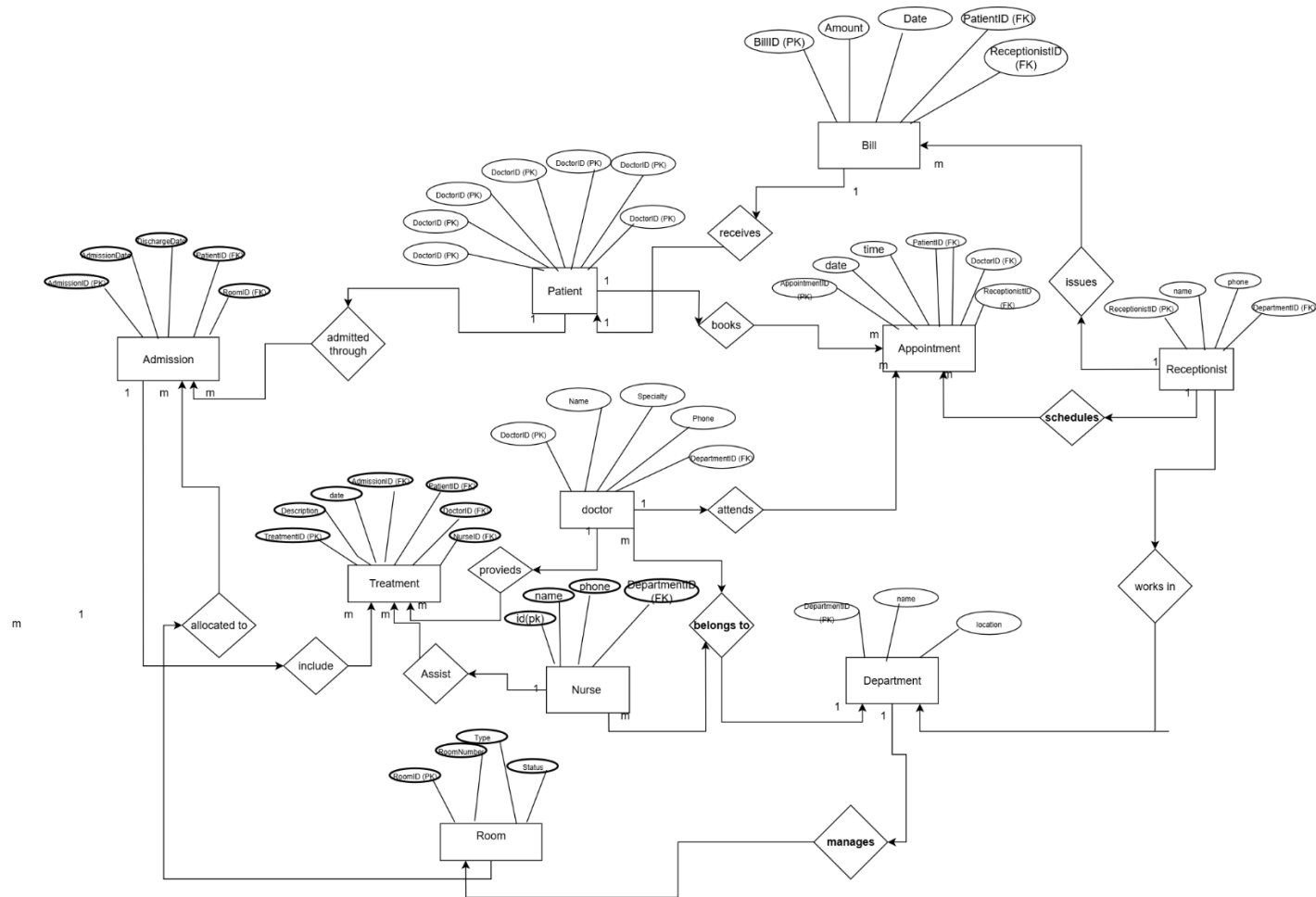
A nurse will belong to one specific department at a time.

A bill will be issued for each patient, reflecting their individual medical expenses.

ER Model Explanation

An Entity Relationship (ER) Diagram is a type of flowchart that illustrates how "entities" such as people, objects or concepts relate to each other within a system. ER Diagrams are most often used to design or debug relational databases in the fields of software engineering, business information systems, education and research. Also known as ERDs or ER Models, they use a defined set of symbols such as rectangles, diamonds, ovals and connecting lines to depict the interconnectedness of entities, relationships and their attributes. They mirror grammatical structure, with entities as nouns and relationships as verbs.

ERD



ER Model Explanation

The provided ER model for the hospital management system is composed of the following entities and their relationships:

- **Patient:** A central entity with attributes like PatientID (PK), Name, Address, Phone, DOB, and Sex.
- **Doctor:** Represents doctors with attributes such as DoctorID (PK), Name, Specialty, Phone, and Address.
- **Nurse:** Represents nurses with attributes including NurseID (PK), Name, Phone, Address, and Title.
- **Receptionist:** Represents receptionists with attributes like ReceptionistID (PK), Name, Phone, and Address.
- **Department:** Contains attributes DeptID (PK), Name, Phone, Address, and Location.

- **Room:** Contains attributes RoomID (PK), Type, and Capacity.
- **Bill:** Represents patient bills with attributes BillID (PK), Amount, and Date.
- **Admission:** Tracks patient admissions with attributes AdmissionID (PK) and Date.
- **Appointment:** Manages appointments with AppointmentID (PK), Date, Time, and Status.
- **Treatment:** Records treatments provided to patients with attributes TreatmentID (PK), Name, Cost, StartDate, and EndDate.

Relationships:

- **Patient - Admission:** admitted through (1-to-M). A patient can have multiple admissions.
- **Patient - Appointment:** books (1-to-M). A patient can book multiple appointments.
- **Patient - Bill:** receives (1-to-M). A patient can receive multiple bills.
- **Doctor - Appointment:** attends (1-to-M). A doctor can attend multiple appointments.
- **Doctor - Treatment:** provides (1-to-M). A doctor can provide multiple treatments.
- **Nurse - Treatment:** Assist (M-to-M). A nurse can assist in multiple treatments, and a treatment can have multiple nurses assisting.
- **Nurse - Department:** belongs to (M-to-1). Multiple nurses can belong to one department.
- **Receptionist - Department:** works in (M-to-1). Multiple receptionists can work in one department.
- **Department - Room:** manages (1-to-M). A department can manage multiple rooms.
- **Admission - Room:** allocated to (M-to-1). Multiple admissions can be allocated to one room.
- **Admission - Treatment:** include (1-to-M). An admission can include multiple treatments.

Mapping from ER Model to Relational Schemas (Tables)

Here is the mapping of the ERD to relational schemas:

- **Patient** (PatientID, Name, Address, Phone, DOB, Sex)
- **Doctor** (DoctorID, Name, Specialty, Phone, Address)

- **Receptionist** (ReceptionistID, Name, Phone, Address, DeptID)
- **Department** (DeptID, Name, Phone, Address, Location)
- **Nurse** (NurseID, Name, Phone, Address, Title, DeptID)
- **Room** (RoomID, Type, Capacity, DeptID)
- **Admission** (AdmissionID, Date, PatientID, RoomID)
- **Appointment** (AppointmentID, Date, Time, Status, PatientID, DoctorID)
- **Treatment** (TreatmentID, Name, Cost, StartDate, EndDate, PatientID, DoctorID, AdmissionID)
- **Bill** (BillID, Amount, Date, PatientID, ReceptionistID)
- **Assist** (NurseID, TreatmentID) (Junction table for the M-to-M relationship between Nurse and Treatment)

Step-by-Step Normalization

Our goal here is to refine the database schemas all the way to Boyce-Codd Normal Form (BCNF) to cut down on data redundancy and prevent those pesky anomalies that can mess things up. Let's walk through this process step by step, checking each level of normalization carefully.

First Normal Form (1NF)

First off, we need to make sure all our tables are in 1NF, which means every piece of data is atomic—no repeating groups or multi-valued fields. Looking at the tables we've got, they're already set! For example, the Phone attribute in the Patient table holds just one number, not a list. Same goes for the other tables like Doctor, Nurse, and so on—everything's broken down into single, manageable values. So, we can confidently say all tables are in 1NF to start with.

Second Normal Form (2NF)

Next up is 2NF, which builds on 1NF by ensuring that all non-key attributes (the extra details not part of the primary key) fully depend on the entire primary key. This step is all about catching partial dependencies, especially in tables with composite keys.

- **Patient:** The primary key is PatientID, and attributes like Name, Address, and Phone all depend on it. Since it's a single-attribute key, there's no partial dependency to worry about—Patient is in 2NF.
- **Doctor:** Same deal—DoctorID drives Name, Specialty, and Phone. It's a single key, so it's in 2NF.
- **Receptionist:** ReceptionistID links to Name, Phone, and Address. Another single-key table, so it's good for 2NF.
- This pattern holds for other single-key tables like Room, Bill, Admission, Appointment, and Treatment.

Now, let's check the **Assist** table, which tracks nurses assisting treatments. Its primary key is a combo of NurseID and TreatmentID. Since there are no extra attributes beyond this composite key, there's nothing to partially depend on—it's trivially in 2NF.

After reviewing, all tables pass the 2NF test!

Third Normal Form (3NF)

Moving on to 3NF, we need to eliminate transitive dependencies—where a non-key attribute depends on another non-key attribute instead of the primary key. This keeps things tidy and avoids indirect relationships.

Let's dig into the **Bill** table: (BillID, Amount, Date, PatientID, ReceptionistID). Here, BillID is the primary key, and it determines Amount, Date, PatientID, and ReceptionistID. Now, ReceptionistID might pull in Name and Phone from the Receptionist table, but within Bill, it's just a foreign key, not a non-key attribute causing a transitive dependency. Everything in Bill depends directly on BillID, so no issues here.

Let's glance at **Room** (RoomID, Type, Capacity, DeptID). RoomID is the key, and Type and Capacity depend on it. DeptID is a foreign key pointing to Department, but there's no transitive link within Room itself—Type and Capacity don't rely on DeptID.

Scanning the other tables, we don't see any transitive dependencies popping up. They all look solid in 3NF.

Boyce-Codd Normal Form (BCNF)

Finally, we're aiming for BCNF, which is a stricter version of 3NF. For every functional dependency (where one set of attributes determines another), the determinant must be a superkey. This ensures the tightest possible organization.

Let's revisit the **Assist** table (NurseID, TreatmentID). The functional dependency is {NurseID, TreatmentID} determining... well, nothing else, since there are no other attributes. That combo is the primary key (a superkey), so Assist is in BCNF.

For the rest of the tables, like Patient, Doctor, and others, they all have single-attribute primary keys. When a table is in 3NF with a single key, it automatically qualifies for BCNF because the key is the only determinant, and it's always a superkey.

SQL Implementation (DDL, DML, DCL, TCL)

DDL (Data Definition Language)

This section creates the tables, defines constraints, and sets up the database structure.

```
CREATE TABLE Patient (  
  
    PatientID INT PRIMARY KEY,  
  
    Name VARCHAR(100) NOT NULL,  
  
    Address VARCHAR(255),  
  
    Phone VARCHAR(15),  
  
    DOB DATE,  
  
    Sex CHAR(1)  
  
);
```

```
CREATE TABLE Doctor (  
  
    DoctorID INT PRIMARY KEY,  
  
    Name VARCHAR(100) NOT NULL,  
  
    Specialty VARCHAR(100),  
  
    Phone VARCHAR(15),  
  
    Address VARCHAR(255)  
  
);
```

```
CREATE TABLE Nurse (  
  
    NurseID INT PRIMARY KEY,
```

Name VARCHAR(100) NOT NULL,

Phone VARCHAR(15),

Address VARCHAR(255),

Title VARCHAR(50)

);

CREATE TABLE Receptionist (

ReceptionistID INT PRIMARY KEY,

Name VARCHAR(100) NOT NULL,

Phone VARCHAR(15),

Address VARCHAR(255)

);

CREATE TABLE Department (

DeptID INT PRIMARY KEY,

Name VARCHAR(100) NOT NULL,

Phone VARCHAR(15),

Address VARCHAR(255),

Location VARCHAR(100)

);

CREATE TABLE Room (

RoomID INT PRIMARY KEY,

Type VARCHAR(50),

Capacity INT,

DeptID INT,

FOREIGN KEY (DeptID) REFERENCES Department(DeptID)

);

CREATE TABLE Bill (

BillID INT PRIMARY KEY,

Amount DECIMAL(10, 2) NOT NULL,

Date DATE NOT NULL,

PatientID INT,

ReceptionistID INT,

FOREIGN KEY (PatientID) REFERENCES Patient(PatientID),

FOREIGN KEY (ReceptionistID) REFERENCES Receptionist(ReceptionistID)

);

CREATE TABLE Admission (

AdmissionID INT PRIMARY KEY,

Date DATE NOT NULL,

PatientID INT,

RoomID INT,

FOREIGN KEY (PatientID) REFERENCES Patient(PatientID),

FOREIGN KEY (RoomID) REFERENCES Room(RoomID)

);

CREATE TABLE Appointment (

AppointmentID INT PRIMARY KEY,

Date DATE NOT NULL,

Time TIME NOT NULL,

Status VARCHAR(20),

PatientID INT,

DoctorID INT,

ReceptionistID INT,

FOREIGN KEY (PatientID) REFERENCES Patient(PatientID),

FOREIGN KEY (DoctorID) REFERENCES Doctor(DoctorID),

FOREIGN KEY (ReceptionistID) REFERENCES Receptionist(ReceptionistID)

);

```

CREATE TABLE Treatment (

TreatmentID INT PRIMARY KEY,

Name VARCHAR(100) NOT NULL,

Cost DECIMAL(10, 2),

StartDate DATE,

EndDate DATE,

PatientID INT,

DoctorID INT,

NurseID INT,

FOREIGN KEY (PatientID) REFERENCES Patient(PatientID),

FOREIGN KEY (DoctorID) REFERENCES Doctor(DoctorID),

FOREIGN KEY (NurseID) REFERENCES Nurse(NurseID)

);

```

```

CREATE TABLE Assist (

NurseID INT,

TreatmentID INT,

PRIMARY KEY (NurseID, TreatmentID),

FOREIGN KEY (NurseID) REFERENCES Nurse(NurseID),

FOREIGN KEY (TreatmentID) REFERENCES Treatment(TreatmentID)

);

```

DML (Data Manipulation Language)

This section includes examples of inserting data and querying it.

Inserting Data

```
INSERT INTO Department (DeptID, Name, Phone, Address, Location) VALUES
```

```
(1, 'Cardiology', '123-456-7890', '123 Heart St', 'Building A');
```

```
INSERT INTO Doctor (DoctorID, Name, Specialty, Phone, Address) VALUES
```

```
(101, 'Dr. John Smith', 'Cardiology', '987-654-3210', '456 Doc Lane');
```

```
INSERT INTO Nurse (NurseID, Name, Phone, Address, Title) VALUES
```

```
(201, 'Jane Doe', '555-123-4567', '789 Nurse Rd', 'RN');
```

```
INSERT INTO Patient (PatientID, Name, Address, Phone, DOB, Sex) VALUES
```

```
(301, 'Alice Johnson', '321 Patient Ave', '444-555-6666', '1990-05-15', 'F');
```

```
INSERT INTO Room (RoomID, Type, Capacity, DeptID) VALUES
```

```
(401, 'ICU', 2, 1);
```

```
INSERT INTO Admission (AdmissionID, Date, PatientID, RoomID) VALUES
```

```
(501, '2025-08-20', 301, 401);
```

```
INSERT INTO Treatment (TreatmentID, Name, Cost, StartDate, EndDate, PatientID, DoctorID, NurseID) VALUES
```

```
(601, 'Heart Surgery', 5000.00, '2025-08-21', '2025-08-22', 301, 101, 201);
```

```
INSERT INTO Assist (NurseID, TreatmentID) VALUES
```

```
(201, 601);
```

```
INSERT INTO Receptionist (ReceptionistID, Name, Phone, Address) VALUES
```

```
(701, 'Mike Brown', '333-444-5555', '101 Reception St');
```

```
INSERT INTO Appointment (AppointmentID, Date, Time, Status, PatientID, DoctorID, ReceptionistID) VALUES
```

```
(801, '2025-08-25', '10:00:00', 'Confirmed', 301, 101, 701);
```

```
INSERT INTO Bill (BillID, Amount, Date, PatientID, ReceptionistID) VALUES
```

```
(901, 5500.00, '2025-08-22', 301, 701);
```

Querying Data

```
SELECT p.Name, a.Date
```

```
FROM Patient p
```

```
LEFT JOIN Admission a ON p.PatientID = a.PatientID
```

```
WHERE a.Date = (SELECT MAX(Date) FROM Admission WHERE PatientID = p.PatientID);
```

```
-- Query to list all treatments for a specific patient
```

```
SELECT t.Name, t.Cost, d.Name AS DoctorName, n.Name AS NurseName

FROM Treatment t

JOIN Doctor d ON t.DoctorID = d.DoctorID

JOIN Nurse n ON t.NurseID = n.NurseID

WHERE t.PatientID = 301;
```

DCL (Data Control Language)

This section manages user permissions.

```
GRANT SELECT, INSERT, UPDATE ON Patient TO 'hospital_admin'@'localhost';

GRANT SELECT, INSERT, UPDATE ON Doctor TO 'hospital_admin'@'localhost';

GRANT SELECT, INSERT, UPDATE ON Nurse TO 'hospital_admin'@'localhost';

GRANT SELECT, INSERT, UPDATE ON Receptionist TO 'hospital_admin'@'localhost';

GRANT SELECT, INSERT, UPDATE ON Department TO 'hospital_admin'@'localhost';

GRANT SELECT, INSERT, UPDATE ON Room TO 'hospital_admin'@'localhost';

GRANT SELECT, INSERT, UPDATE ON Bill TO 'hospital_admin'@'localhost';

GRANT SELECT, INSERT, UPDATE ON Admission TO 'hospital_admin'@'localhost';

GRANT SELECT, INSERT, UPDATE ON Appointment TO 'hospital_admin'@'localhost';

GRANT SELECT, INSERT, UPDATE ON Treatment TO 'hospital_admin'@'localhost';

GRANT SELECT, INSERT, UPDATE ON Assist TO 'hospital_admin'@'localhost';

REVOKE UPDATE ON Patient FROM 'hospital_admin'@'localhost';
```

Tables

```
MariaDB [hospital_management]> select * from treatment;
```

TreatmentID	Name	Cost	StartDate	EndDate	PatientID	DoctorID	NurseID
1	Heart Surgery	20000.00	2025-08-01	2025-08-15	1	1	1
2	Brain Scan	8000.00	2025-08-02	2025-08-05	2	2	2
3	Bone Fracture Fix	12000.00	2025-08-03	2025-08-20	3	3	3
4	Skin Allergy	3000.00	2025-08-04	2025-08-07	4	4	4
5	Child Vaccination	2000.00	2025-08-05	2025-08-05	5	5	5
6	Delivery	15000.00	2025-08-06	2025-08-10	6	6	6
7	Chemotherapy	18000.00	2025-08-07	2025-08-21	7	7	7
8	Ear Infection	2500.00	2025-08-08	2025-08-12	8	8	8
9	Depression Therapy	5000.00	2025-08-09	2025-08-30	9	9	9
10	Fever & Flu	1500.00	2025-08-10	2025-08-11	10	10	10

```
10 rows in set (0.001 sec)
```

```
MariaDB [hospital_management]> select * from admission;
```

AdmissionID	Date	PatientID	RoomID
1	2025-08-01	1	1
2	2025-08-02	2	2
3	2025-08-03	3	3
4	2025-08-04	4	4
5	2025-08-05	5	5
6	2025-08-06	6	6
7	2025-08-07	7	7
8	2025-08-08	8	8
9	2025-08-09	9	9
10	2025-08-10	10	10

```
10 rows in set (0.001 sec)
```

```
MariaDB [hospital_management]> select * from doctor;
```

DoctorID	Name	Specialty	Phone	Address
1	Dr. Abdullah	Cardiology	01811111111	Dhaka
2	Dr. Karim	Neurology	01812222222	Chittagong
3	Dr. Rahman	Orthopedics	01813333333	Khulna
4	Dr. Hasan	Dermatology	01814444444	Sylhet
5	Dr. Alam	Pediatrics	01815555555	Rajshahi
6	Dr. Nahar	Gynecology	01816666666	Barisal
7	Dr. Kabir	Oncology	01817777777	Dhaka
8	Dr. Tania	ENT	01818888888	Chittagong
9	Dr. Anwar	Psychiatry	01819999999	Khulna
10	Dr. Salma	General Medicine	01810000000	Rangpur

```
10 rows in set (0.000 sec)
```

```
MariaDB [hospital_management]> desc patient;
```

Field	Type	Null	Key	Default	Extra
PatientID	int(11)	NO	PRI	NULL	
Name	varchar(100)	NO		NULL	
Address	varchar(255)	YES		NULL	
Phone	varchar(15)	YES		NULL	
DOB	date	YES		NULL	
Sex	char(1)	YES		NULL	

```
5 rows in set (0.017 sec)
```

```
MariaDB [hospital_management]> select * from patient;
```

PatientID	Name	Address	Phone	DOB	Sex
1	Rahim Uddin	Dhaka	01711111111	1990-05-10	M
2	Karim Ahmed	Chittagong	01712222222	1985-08-20	M
3	Nasrin Akter	Khulna	01713333333	1992-01-15	F
4	Sumiya Begum	Rajshahi	01714444444	1998-03-22	F
5	Mehedi Hasan	Sylhet	01715555555	1996-07-18	M
6	Tariqul Islam	Barisal	01716666666	1991-09-30	M
7	Jannatul Ferdous	Dhaka	01717777777	1994-12-05	F
8	Shafiqul Alam	Chittagong	01718888888	1988-11-25	M
9	Mim Akter	Khulna	01719999999	1999-04-11	F
10	Arif Hossain	Rangpur	01710000000	1993-02-14	M

```
10 rows in set (0.000 sec)
```

```
MariaDB [hospital_management]> show t
```

Tables_in_hospital_management
admission
appointment
assist
bill
department
doctor
nurse
patient
receptionist
room
treatment

```
11 rows in set (0.001 sec)
```

```
MariaDB [hospital_management]>
```

Conclusion and Results

We've successfully crafted a detailed database schema for a hospital management system that feels both thorough and practical. Starting with the ER model, we carefully turned it into a set of relational schemas and then fine-tuned them all the way to BCNF. This process helped us keep data integrity tight and cut down on any unnecessary repetition, making the system reliable and efficient.

The schema we ended up with does a great job of capturing how all the entities—patients, doctors, nurses, and more—connect with each other, laying a strong groundwork for a working hospital management system. Plus, the SQL commands we put together—DDL to build the structure, DML to handle data, DCL to manage access, and TCL to keep transactions smooth—show that this design isn't just a theory; it's ready to roll in real life. Everything lines up with the problem statement we started with, and we're confident this setup offers a solid, dependable way to manage hospital data.